Name : AKASH SAMPATH

Restart Concepts : (Types) :

restart: no	• Default behaviour • If container stops , docker does not restart it
restart: always	• Docker always restarts container. • Even if you manually stop it . • Docker start it again automatically
restart: unless-stopped	• Restart automatically after crash • Restart after system reboot • Restart after Docker restart • So If manually stop container , Docker remembers and wont auto start it

Why two environment needed :

- Each container is like a separate computer :

mysql: environment:	app: environment:
This container needs is own settings	This spring boot app needs its own settings

- So , that's why both services have their own environment

Matching : (In .env file) (docker compose yml) : (Both)

SPRING_DATASOURCE_USERNAME : akash	MYSQL_USER : akash
SPRING_DATASOURCE_PASSWORD : akash123	MYSQL_PASSWORD : akash123

So therefore , Two environments are connected :

MYSQL Creates :	MYSQL_USER : akash MYSQL_PASSWORD : akash123
Application Uses :	SPRING_DATASOURCE_USERNAME : akash SPRING_DATASOURCE_PASSWORD : akash123

Using variables in both places : In . env file : In docker compose yml :

Because :

- In . env file we store values which can be reuseable
- In docker compose , Use the values to run containers

Based on current setup : We have same values written twice :

In . env file :

MYSQL_DATABASE = electricity_db
MYSQL_USER = akash
MYSQL_PASSWORD = akash123

So again in docker compose : environment :

MYSQL_DATABASE : electricity_db
MYSQL_USER : akash
MYSQL_PASSWORD : akash123

- So docker is not actually using . env , We are using the values directly inside docker compose file based on . env values

Practices :

Not best practice : Because duplication causes problems :

- If we change the password in . env file :

MYSQL_PASSWORD = devops

- But forget to change in docker compose file :

MYSQL_PASSWORD : akash123

- Now . env says one password and docker compose says another password
- Finally application fails

Best practice : Store values only in . env and then use in docker compose .yml :

In . env file :

MYSQL_DATABASE = electricity_db
MYSQL_USER = akash
MYSQL_PASSWORD = akash123
MYSQL_ROOT_PASSWORD = root123
SPRING_DATASOURCE_URL = jdbc:mysql://mysql:3306/electricity_db
SPRING_DATASOURCE_USERNAME = akash
SPRING_DATASOURCE_PASSWORD = akash123

Use these variables in docker compose yml :

environment:
MYSQL_ROOT_PASSWORD : $ { MYSQL_ROOT_PASSWORD }
MYSQL_DATABASE : $ { MYSQL_DATABASE }
MYSQL_USER : $ { MYSQL_USER }
MYSQL_PASSWORD : $ { MYSQL_PASSWORD }

environment:
SPRING_DATASOURCE_URL : $ { SPRING_DATASOURCE_URL }
SPRING_DATASOURCE_USERNAME : $ { SPRING_DATASOURCE_USERNAME }
SPRING_DATASOURCE_PASSWORD : $ { SPRING_DATASOURCE_PASSWORD }

So therefore , $ { VARIABLE_NAME } means :

MYSQL _ USER : $ { MYSQL_USER }

- We take values from . env file and docker automatically reads . env file

So can we use only . env :

- No , because . env stores variables
- It doesn't not create containers
- Docker compose uses them

So why use both :

- So . env stores values and docker compose uses them
- We can use only compose and cannot use only . env
- So just store in . env and use in compose
- . env is the storage which holds value and docker compose is the instructions which uses the value

. env and env Concepts :

- So . env is a special filename used automatically by docker compose
- So this file name is exactly as . env
- So docker uses . env as docker compose automatically looks for . env in the same folder as docker compose yml
- Docker finds it by reading all variables automatically and you can use variables inside docker compose

Example :

```
environment :
        MYSQL_DATABASE : $ { MYSQL_DATABASE }
```

Docker replaces :

```
$ { MYSQL_DATABASE }
```

With : from : (. env)

```
electricity_db
```

Suppose in . env :

```
MYSQL_USER = akash
```

Then in docker compose :

```
environment:
        MYSQL_USER: $ { MYSQL_USER }
```

So when docker runs using docker compose up :

- The docker converts it into :

```
environment:
        MYSQL_USER = akash
```

(.) dot is important in .env :

- It is the standard environment file name
- It is auto recognized by docker compose

So without dot in env :

- Docker does not auto load it

Example : If we use env instead of . env :

- Docker compose will not automatically read it

So this :

```
MYSQL_USER : $ { MYSQL_USER }
```

becomes :

```
MYSQL_USER : ( empty value ) ( variable not set )
```

So using . env :

- Files such as docker compose use . env which docker auto loads variables and can directly use :

```
$ { MYSQL_USER }
```

So using env :

- Files such as docker compose yml , Docker ignores it and variables are not loaded automatically

Also we can use another file name :

Example : akash . env

- So we must specify manually : docker compose --env-file akash .env up
- So now docker loads it

So ,

. env files stores value or variables in environment and in compose sends variables into container :

```
environment:
        MYSQL_USER : $ { MYSQL_USER }
```

So this means , take MYSQL_USER from . env and pass it into container

- So using . env : From . env docker compose reads it automatically and variables are inserted in into docker compose and the container gets its environment variables